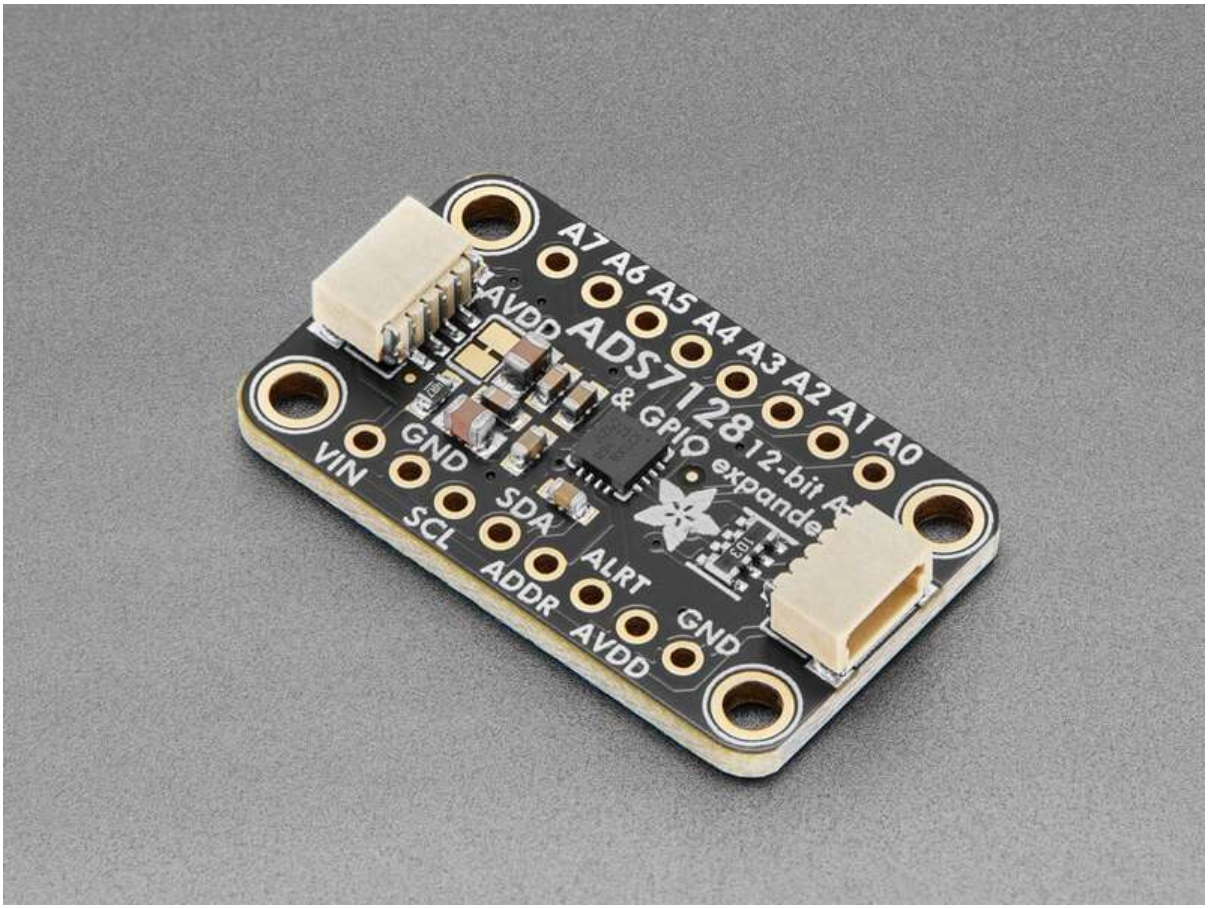
Adafruit ADS7128 8-Channel ADC and GPIO Expander

Created by Liz Clark



https://learn.adafruit.com/adafruit-ads7128-8-channel-adc-and-gpio-expander

Last updated on 2026-07-02 08:19:21 PM UTC

Table of Contents

Overview 3

Pinouts 5

- Power Pins
- I2C Logic Pins
- ADC/GPIO Expander Pins
- Address Pin and Jumpers
- Interrupt Pin
- Power LED and Jumper

CircuitPython and Python 7

- CircuitPython Microcontroller Wiring
- Python Computer Wiring
- Python Installation of ADS7128 Library
- CircuitPython Usage
- Python Usage
- Example Code

Python Docs 13

Arduino 13

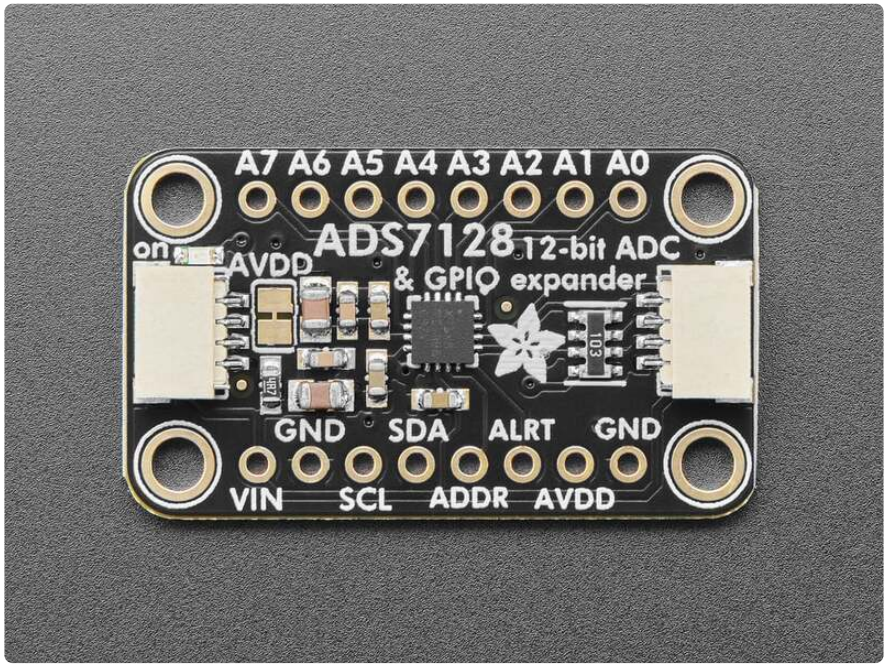
- Wiring
- Library Installation
- Example Code

Arduino Docs 17

Downloads 18

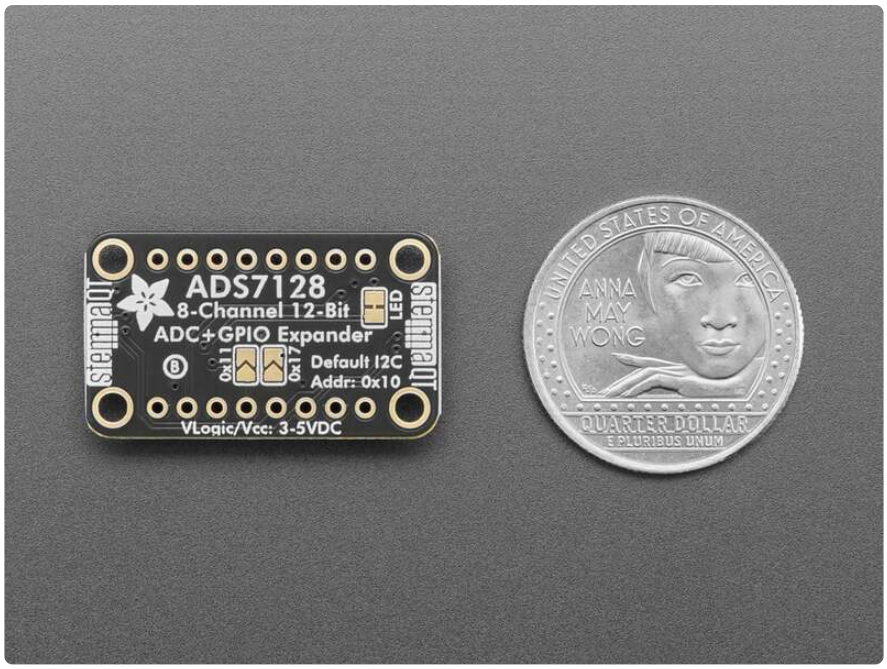
- Files
- Schematic and Fab Print
- 3D Model

Overview



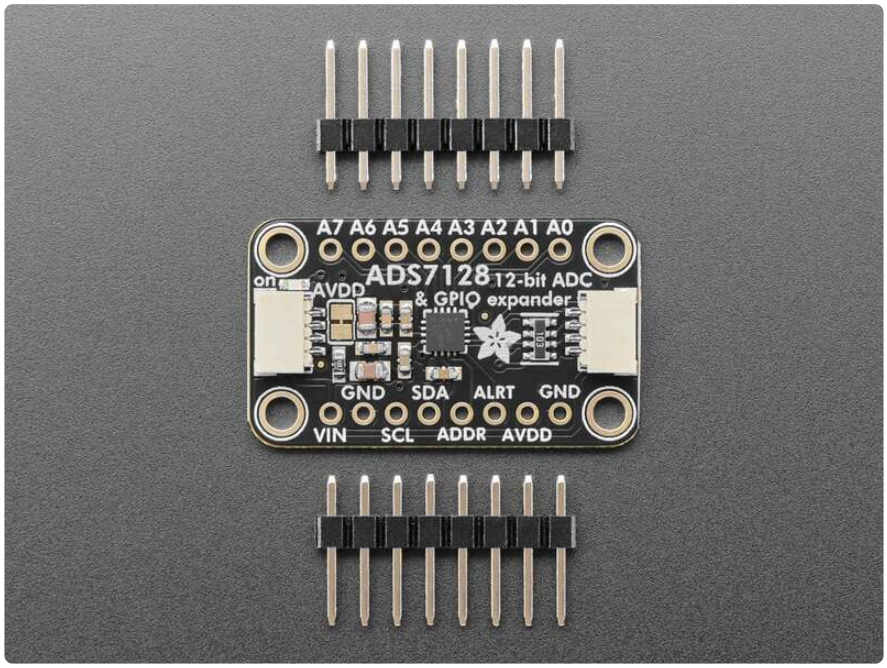
The **Adafruit ADS7128 8-Channel ADC and GPIO Expander** is a bit of a Platypus of the I2C expander world: hard to categorize as just an ADC or a GPIO expander since it combines the capabilities of both with great capabilities! Usually, you only get ADC or I/O, so it's nice to see this combo product.

Featuring the aforementioned ADS7128, it can run on 3.3 or 5V logic and power, and all 8 I/O pins can individually configured to perform either 12-bit SAR analog input, push-pull or open drain digital output, or digital input. This makes it great as a general-purpose glue chip when you want to add more pins or analog/open-drain outputs to a microcontroller or microcomputer.



The ADS7128 also has a lot of fancy extras that make it a good ADC companion even if your main core already has analog inputs. It can run in standalone mode with zero-crossing detection or RMS computation. Not only can it count and track threshold crossings, but it also has an alert trigger output.

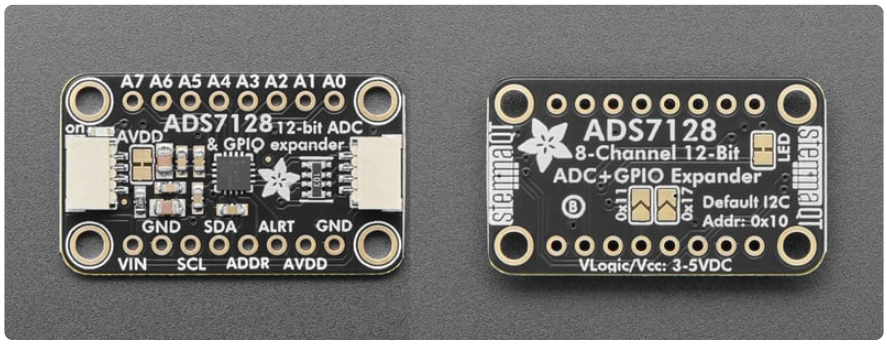
Unlike many ADC/expanders, it has two power supplies: the DVDD digital supply and AVDD analog supply are separate, so you can use 3V logic for the digital I/O and I2C communications while the analog inputs and digital outputs can be as high as 5V. Or vice versa!



Interfacing is easy - this sensor uses plain, universal I2C. We put this sensor on a breakout board with all the extras needed so you can use it with 3.3V or 5V power/logic microcontrollers. We have written libraries for Arduino (C/C++) as well as CircuitPython (Python 3) (https://adafru.it/1aF2)so you can use this sensor with just about any kind of device, even a Raspberry Pi!

As if that weren't enough, we've also added SparkFun qwiic (https://adafru.it/Fpw) compatible STEMMA QT **(https://adafru.it/Ft4)** connectors for the I2C bus **so you don't even need to solder.** Just wire up to your favorite micro with a plug-and-play cable to get analog inputs and digital I/Os ASAP. For a no-solder experience, just wire up to your favorite micro, like the ESP32 V2 Feather (http://adafru.it/5400) using a STEMMA QT adapter cable (https://adafru.it/JnB). The Stemma QT connectors also mean the ADS7128 can be used with our various associated accessories (https://adafru.it/Ft6). QT Cable is not included, but we have a variety in the shop (https://adafru.it/17VE).

Pinouts



The default I2C address is **0x10**.

Power Pins

- **VIN** - this is the power pin. Since the ADC chip uses 3-5 VDC to power the board, give it the same power as the logic level of your microcontroller - e.g. for a 5V micro like Arduino, use 5V.
- **AVDD** - this is the ADC reference voltage pin. By default it is connected to VIN and the ADC will use the voltage level of VIN.
- **AVDD Jumper** - On the left side of the front of board is the **AVDD jumper**. If you cut this jumper, it disconnects **AVDD** from **VIN**. You can then connect the **AVDD** pin to a different voltage level between 3-5 VDC for the ADC.
- **GND** - common ground for power and logic.

I2C Logic Pins

- **SCL** - I2C clock pin, connect to your microcontroller I2C clock line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.
- **SDA** - I2C data pin, connect to your microcontroller I2C data line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.
- STEMMA QT **(https://adafru.it/Ft4)** - These connectors allow you to connectors to dev boards with **STEMMA QT** connectors or to other things with various associated accessories (https://adafru.it/Ft6)

ADC/GPIO Expander Pins

- **A0-A7** - The 8 pins to the ADC and GPIO expander. Each pin can be configured as either an analog input, digital input or digital output. The ADS7128 has no internal pull resistors, so you will want to add a pull-up or pull-down resistor when using a pin as an digital input.

Text emphasized with a yellow info:

The ADS7128 has no internal pull resistors, so you will want to add a pull-up or pull-down resistor when using a pin as an digital input.

Address Pin and Jumpers

On the front of the board is the I2C address pin, labeled **ADDR**. The I2C address is set by connecting external resistors to the ADDR pin. The address is set based on the resistor values at power-up. The table below is Table 2 from the ADS7218 datasheet and shows the resistor values and the corresponding addresses.

Table 2. I²C Address Selection

RESISTORS		ADDRESS
R1 (1)	R2 (1)	
0 Ω	DNP (2)	001 0111b (17h)
11 kΩ	DNP (2)	001 0110b (16h)
33 kΩ	DNP (2)	001 0101b (15h)
100 kΩ	DNP (2)	001 0100b (14h)
DNP (2)	DNP (2)	001 0000b (10h)
DNP (2)	11 kΩ	001 0001b (11h)
DNP (2)	33 kΩ	001 0010b (12h)
DNP (2)	100 kΩ	001 0011b (13h)

(1) Tolerance for R1, R2 ≤ ±5%.
(2) DNP = Do not populate.

On the back of the board are **two address jumpers**, labeled **0x11** and **0x17**. These jumpers allow you to set the I2C address to either **0x11** or **0x17**. To do so, you solder the jumper "closed" by connecting the two pads.

Interrupt Pin

- **ALRT** - This is the interrupt pin. By default it is an open-drain output.

Power LED and Jumper

- **Power LED -** In the upper left corner, above the STEMMA connector, on the front of the board, is the power LED, labeled **on**. It is the green LED.
- **LED jumper** - In the upper right corner on the back of the board is a jumper for the power LED. If you wish to disable the power LED, cut the trace on this jumper.

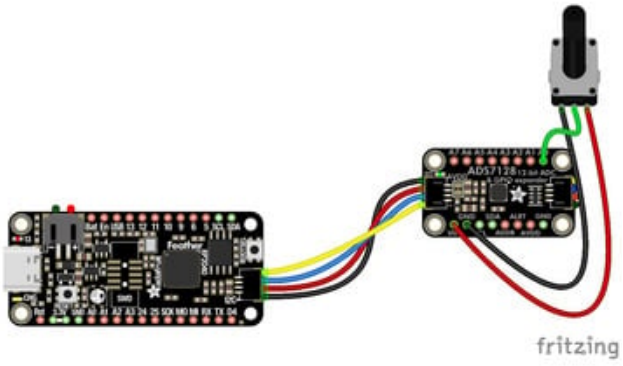
CircuitPython and Python

It's easy to use the **ADS7128** with Python or CircuitPython, and the Adafruit_CircuitPython_ADS7128 (https://adafru.it/1aF3) module. This module allows you to easily write Python code to read the 8 analog inputs, digital inputs or digital outputs on the expander.

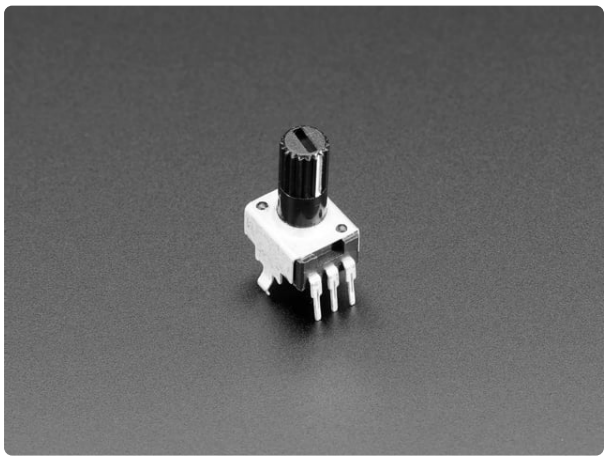
You can use this driver with any CircuitPython microcontroller board or with a computer that has GPIO and Python thanks to Adafruit_Blinka, our CircuitPython-for-Python compatibility library (https://adafru.it/BSN).

CircuitPython Microcontroller Wiring

First wire up the breakout to your board exactly as follows. The following is the breakout wired to a Feather RP2040 using the STEMMA connector along with a potentiometer connected to analog input 0 (**A0**) on the ADC:

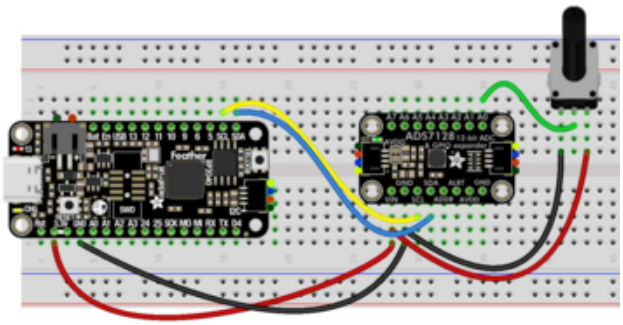


Board STEMMA 3V to **breakout VIN (red wire)**
Board STEMMA GND to **breakout GND (black wire)**
Board STEMMA SCL to **breakout SCL (yellow wire)**
Board STEMMA SDA to **breakout SDA (blue wire)**
Breakout VIN to **potentiometer positive (red wire)**
Breakout A0 to **potentiometer wiper (green wire)**
Breakout GND to **potentiometer negative (black wire)**



Potentiometer with Built In Knob - 10K ohm
Oh say can you seeBy the knob's early light...Sorry - we thought that was clever. And while it wasn't really, this potentiometer definitely...
https://www.adafruit.com/product/4133

The following is the breakout wired to a Feather RP2040 using a solderless breadboard:

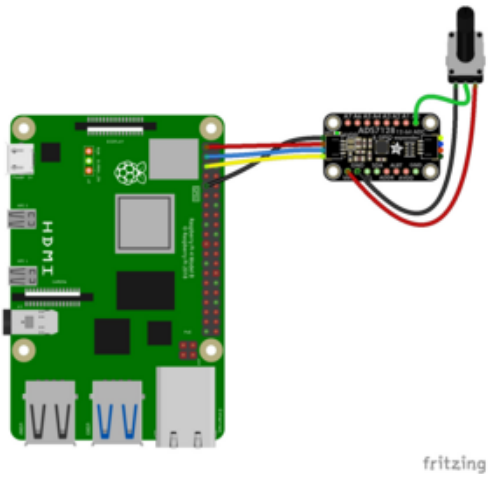


Board 3V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**
Breakout VIN to **potentiometer positive (red wire)**
Breakout A0 to **potentiometer wiper (green wire)**
Breakout GND to **potentiometer negative (black wire)**

Python Computer Wiring

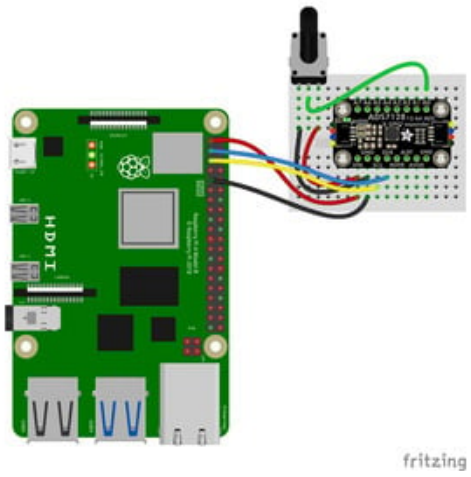
Since there are dozens of Linux computers/boards you can use, we will show wiring for Raspberry Pi. For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported](https://adafru.it/BSN) (https://adafru.it/BSN).

Here's the Raspberry Pi wired with I2C using the STEMMA connector:



Pi 3V to **breakout VIN (red wire)**
Pi GND to **breakout GND (black wire)**
Pi SCL to **breakout SCL (yellow wire)**
Pi SDA to **breakout SDA (blue wire)**
Breakout VIN to **potentiometer positive (red wire)**
Breakout A0 to **potentiometer wiper (green wire)**
Breakout GND to **potentiometer negative (black wire)**

Here's the Raspberry Pi wired with I2C using a solderless breadboard:



Pi 3V to **breakout VIN (red wire)**
Pi GND to **breakout GND (black wire)**
Pi SCL to **breakout SCL (yellow wire)**
Pi SDA to **breakout SDA (blue wire)**
Breakout VIN to **potentiometer positive (red wire)**
Breakout A0 to **potentiometer wiper (green wire)**
Breakout GND to **potentiometer negative (black wire)**

Python Installation of ADS7128 Library

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling I2C on your platform and verifying you are running Python 3. Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready (https://adafru.it/BSN)!

Once that's done, from your command line run the following command:

- `pip3 install adafruit-circuitpython-ads7128`

If your default Python is version 3, you may need to run 'pip' instead. Just make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

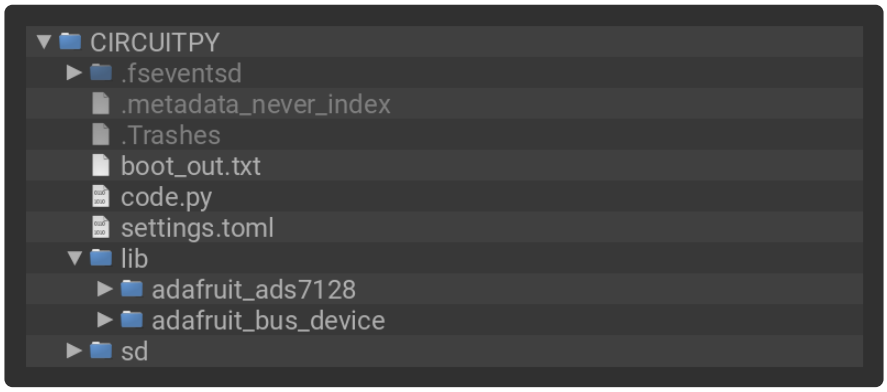
CircuitPython Usage

To use with CircuitPython, you need to first install the **Adafruit_CircuitPython_ADS7128** library, and its dependencies, into the **lib** folder on your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY/lib** folder should contain the following folders:

- **adafruit_bus_device/**
- **adafruit_ads7128/**



Python Usage

Once you have the library `pip3` installed on your computer, copy or download the following example to your computer, and run the following, replacing **code.py** with whatever you named the file:

```
python3 code.py
```

Example Code

If running CircuitPython: Once everything is saved to the **CIRCUITPY** drive, connect to the serial console (https://adafru.it/Bec) to see the data printed out!

If running Python: The console output will appear wherever you are running Python.

```
# SPDX-FileCopyrightText: 2026 Liz Clark for Adafruit Industries
#
# SPDX-License-Identifier: MIT

"""Simple analog read example for the ADS7128.

Reads a single channel and prints the 16-bit value and the voltage.
"""

import time

import board

from adafruit_ads7128.ads7128 import ADS7128
from adafruit_ads7128.analog_in import AnalogIn
```

```
16
17  # Reference voltage
18  REFERENCE_VOLTAGE = 3.3
19
20  i2c = board.I2C()
21  adc = ADS7128(i2c)
22
23  # analog input on channel 0
24  channel = AnalogIn(adc, 0)
25
26  while True:
27      print(f"value: {channel.value}  voltage: {channel.voltage(REFERENCE_VOLTAGE):.3f} V")
28      time.sleep(1.0)
```

https://github.com/adafruit/Adafruit_CircuitPython_ADS7128/blob/main/examples/ads7128_simpletest.py

The ADS7128 is initialized over I2C. Then in the loop, the analog input on A0 is read and printed to the serial monitor. As you adjust the potentiometer connected to A0, you'll see the values change.

Text emphasized with a yellow exclamation:

To make code from different ADC's all compatible, we always return a 16-bit value (0 to 65535) even though, in this case, the underlying hardware is 12-bit!

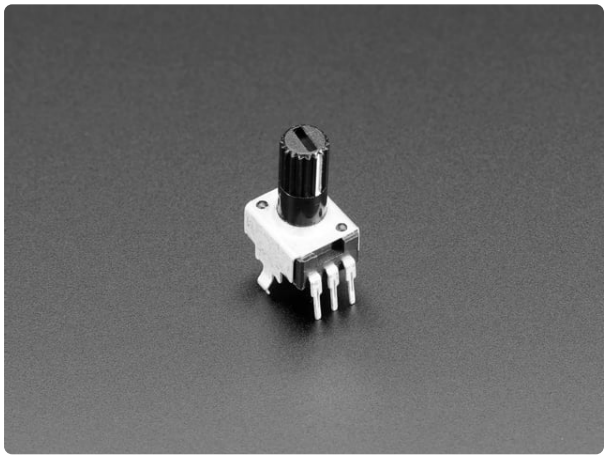
```
value: 0  voltage: 0.001 V
value: 32  voltage: 0.003 V
value: 16723  voltage: 0.850 V
value: 43433  voltage: 2.193 V
value: 54156  voltage: 2.707 V
value: 29846  voltage: 1.511 V
value: 400  voltage: 0.019 V
value: 0  voltage: 0.001 V
value: 20932  voltage: 1.055 V
value: 65535  voltage: 3.299 V
value: 65535  voltage: 3.299 V
value: 65535  voltage: 3.299 V
```

Python Docs

Python Docs (https://adafru.it/1aF0)

Arduino

Using the ADS7128 breakout with Arduino involves wiring up the breakout to your Arduino-compatible microcontroller with a potentiometer, installing the Adafruit_ADS7128 (https://adafru.it/1aF4) library, and running the provided example code.

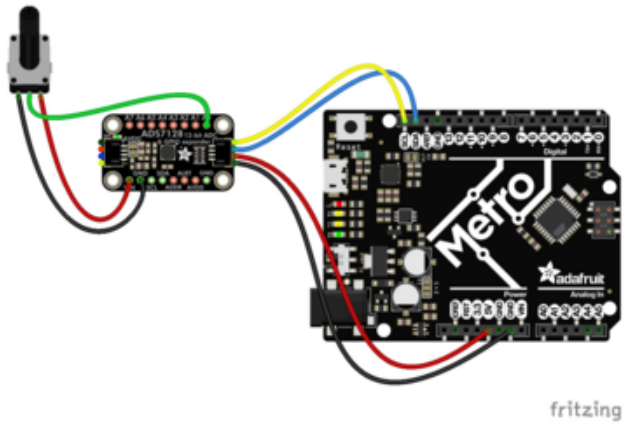


Potentiometer with Built In Knob - 10K ohm
Oh say can you seeBy the knob's early light...Sorry - we thought that was clever. And while it wasn't really, this potentiometer definitely...
https://www.adafruit.com/product/4133

Wiring

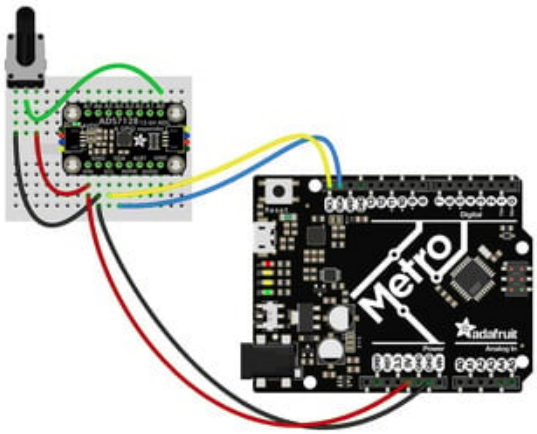
Wire as shown for a **5V** board like an Uno. If you are using a **3V** board, like an Adafruit Feather, wire the board's 3V pin to the breakout VIN.

Here is an Adafruit Metro wired up to the breakout using the STEMMA QT connector. A potentiometer is connected to input 0 (**A0**) on the ADC:



Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**
Breakout VIN to **potentiometer positive (red wire)**
Breakout A0 to **potentiometer wiper (green wire)**
Breakout GND to **potentiometer negative (black wire)**

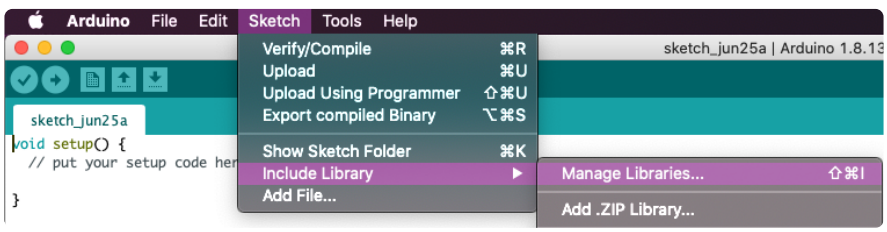
Here is an Adafruit Metro wired up using a solderless breadboard:



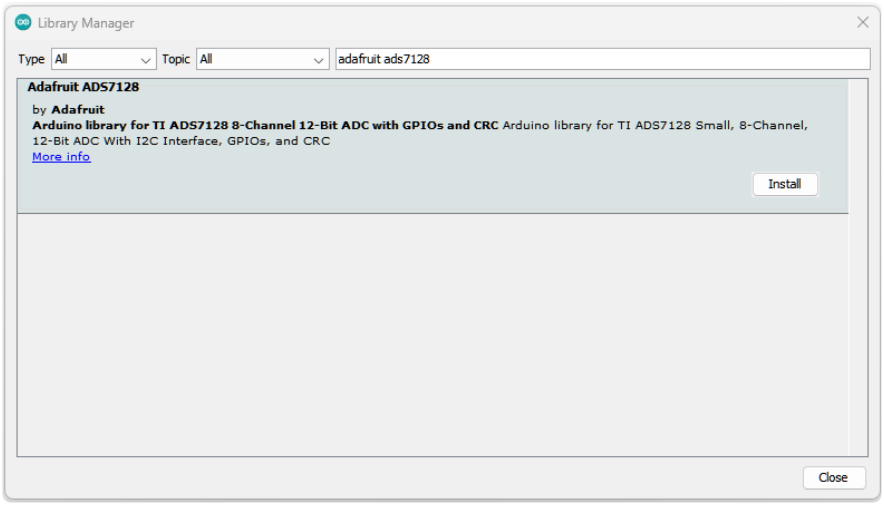
Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**
Breakout VIN to **potentiometer positive (red wire)**
Breakout A0 to **potentiometer wiper (green wire)**
Breakout GND to **potentiometer negative (black wire)**

Library Installation

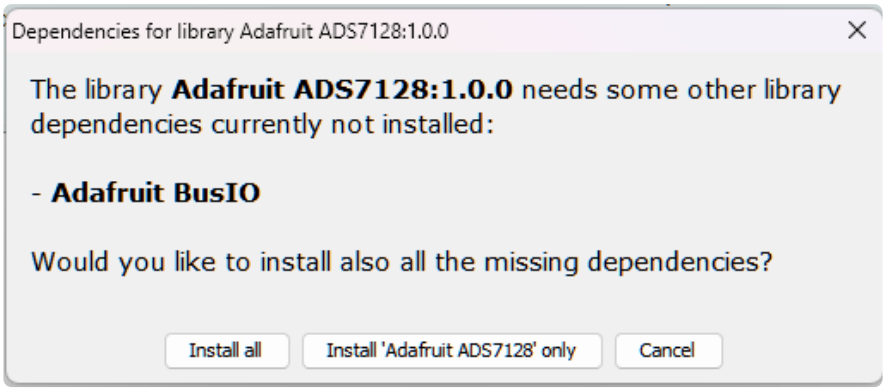
You can install the **Adafruit_ADS7128** library for Arduino using the Library Manager in the Arduino IDE.



Click the **Manage Libraries ...** menu item, search for **Adafruit_ADS7128**, and select the **Adafruit ADS7128** library:



If asked about dependencies, click "Install all".



If the "Dependencies" window does not come up, then you already have the dependencies installed.

Text emphasized with a yellow exclamation:

If the dependencies are already installed, you must make sure you update them through the Arduino Library Manager before loading the example!

Example Code

```
/*!
 * @file 01_adc_simpletest.ino
 * @brief Read all 8 ADC channels in manual mode
 *
 * Reads all 8 channels and prints raw values and voltages.
 * Change VREF below to match your supply voltage.
 */

#include <Adafruit_ADS7128.h>

// Set this to your AVDD voltage (3.3 or 5.0)
#define ADS_VREF 3.3

Adafruit_ADS7128 adc;

void setup() {
  Serial.begin(115200);
  while (!Serial) {
    delay(10);
  }

  Serial.println(F("ADS7128 ADC Simple Test"));
  Serial.println(F("======================="));

  // begin() defaults: address ADS7128_DEFAULT_ADDR (0x10), Wire bus &Wire
  if (!adc.begin(ADS7128_DEFAULT_ADDR, &Wire)) {
    Serial.println(F("Failed to find ADS7128!"));
    while (1) {
      delay(100);
    }
  }

  Serial.println(F("ADS7128 initialized!"));
  Serial.println();
}

void loop() {
  Serial.println(F("Channel  Raw    Voltage"));
  Serial.println(F("-------  ----   -------"));

  for (uint8_t ch = 0; ch < 8; ch++) {
    uint16_t raw = adc.analogRead(ch);
    float voltage = adc.analogReadVoltage(ch, ADS_VREF);

    Serial.print(F("    "));
    Serial.print(ch);
    Serial.print(F("      "));
    if (raw < 1000) {
      Serial.print(F(" "));
    }
    if (raw < 100) {
      Serial.print(F(" "));
    }
    if (raw < 10) {
```

```
      Serial.print(F(" "));
    }
    Serial.print(raw);
    Serial.print(F("    "));
    Serial.print(voltage, 3);
    Serial.println(F(" V"));
  }

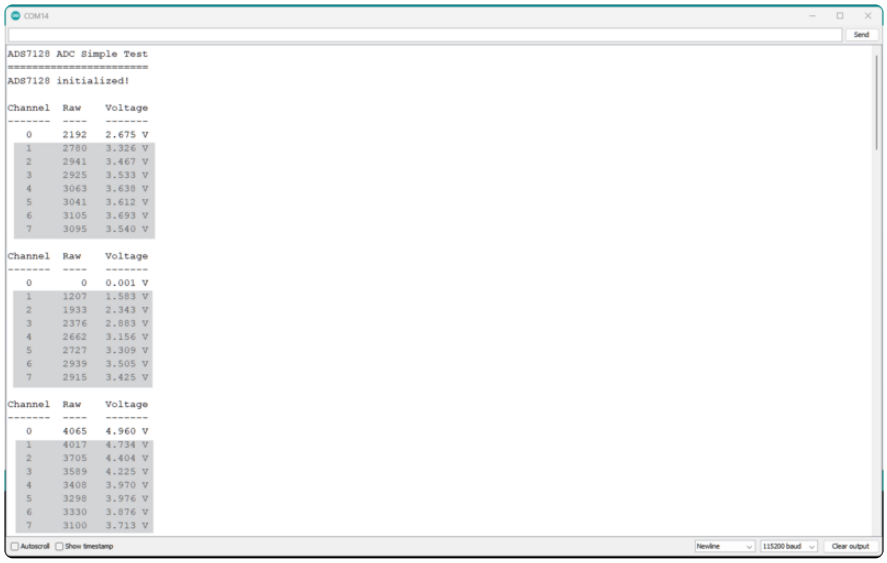
  Serial.println();
  delay(500);
}
```

https://github.com/adafruit/Adafruit_ADS7128/blob/main/examples/01_adc_simpletest/01_adc_simpletest.ino

Upload the sketch to your board and open up the Serial Monitor (**Tools -> Serial Monitor**) at 115200 baud. You'll see the ADS7128 recognized over I2C. As you turn the potentiometer attached to A0, you'll see the raw value and voltage reading change.

Text emphasized with a blue info:

The remaining 7 channels (A1 through A7) are 'floating' - not connected to anything, so you will see varying random values.



Arduino Docs

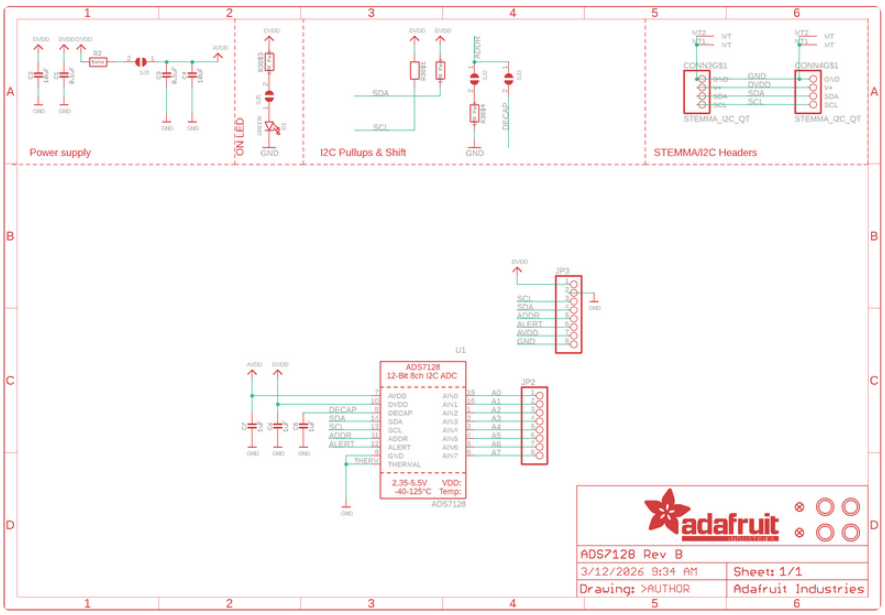
Arduino Docs (https://adafru.it/1aF1)

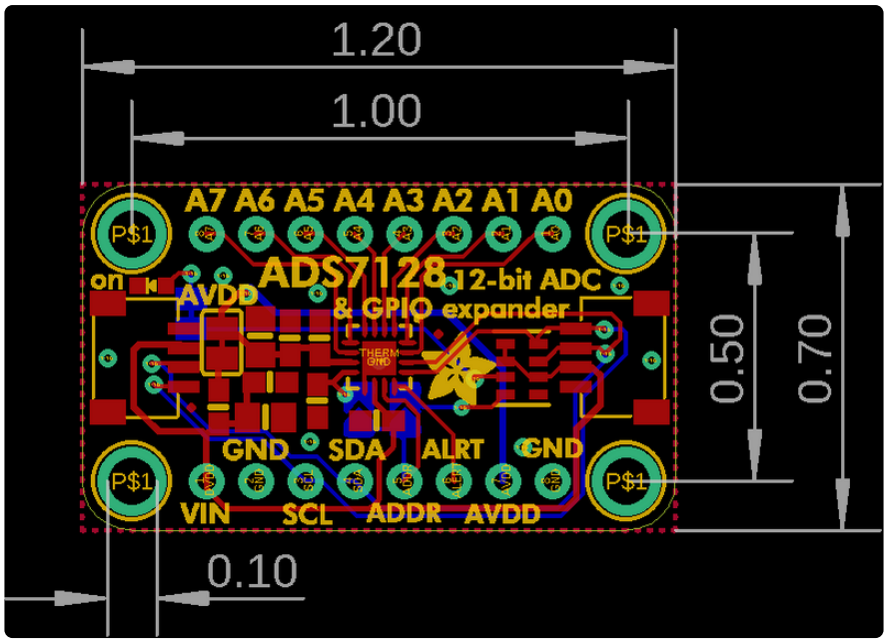
Downloads

Files

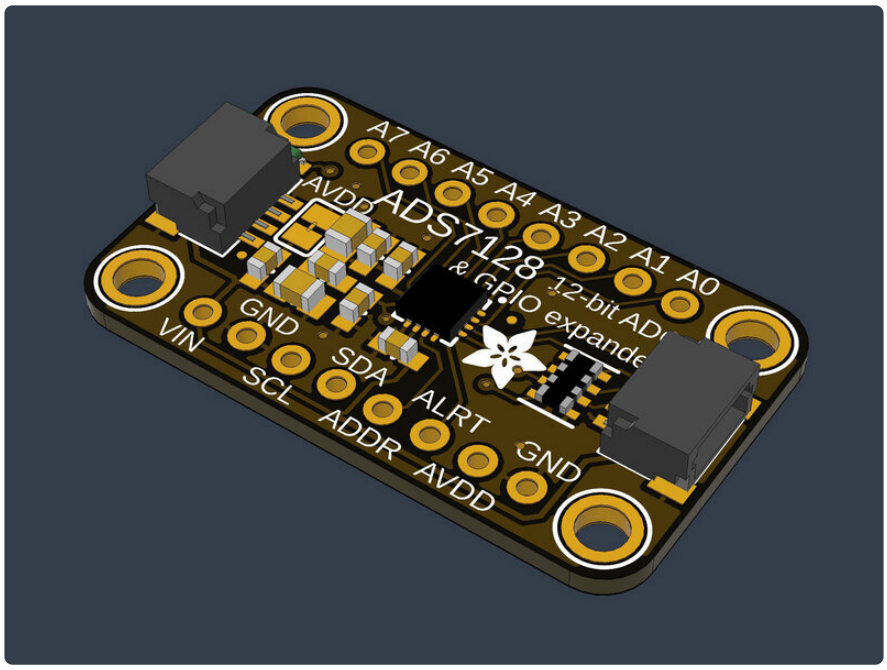
- ADS7128 Datasheet (https://adafru.it/1aF5)
- EagleCAD PCB files on GitHub (https://adafru.it/1aF6)
- 3D models on GitHub (https://adafru.it/1aFe)
- Fritzing object in the Adafruit Fritzing Library (https://adafru.it/1aF7)

Schematic and Fab Print




1.20
1.00
A7 A6 A5 A4 A3 A2 A1 A0
P$1
P$1
ADS7128 12-bit ADC
& GPIO expander
AVDD
0.50
0.70
GND SDA ALRT GND
P$1
P$1
VIN SCL ADDR AVDD
0.10

3D Model


A7 A6 A5 A4 A3 A2 A1 A0
AVDD
ADS7128
12-bit ADC
& GPIO expander
VIN
GND
SCL
SDA
ADDR
ALRT
AVDD
GND